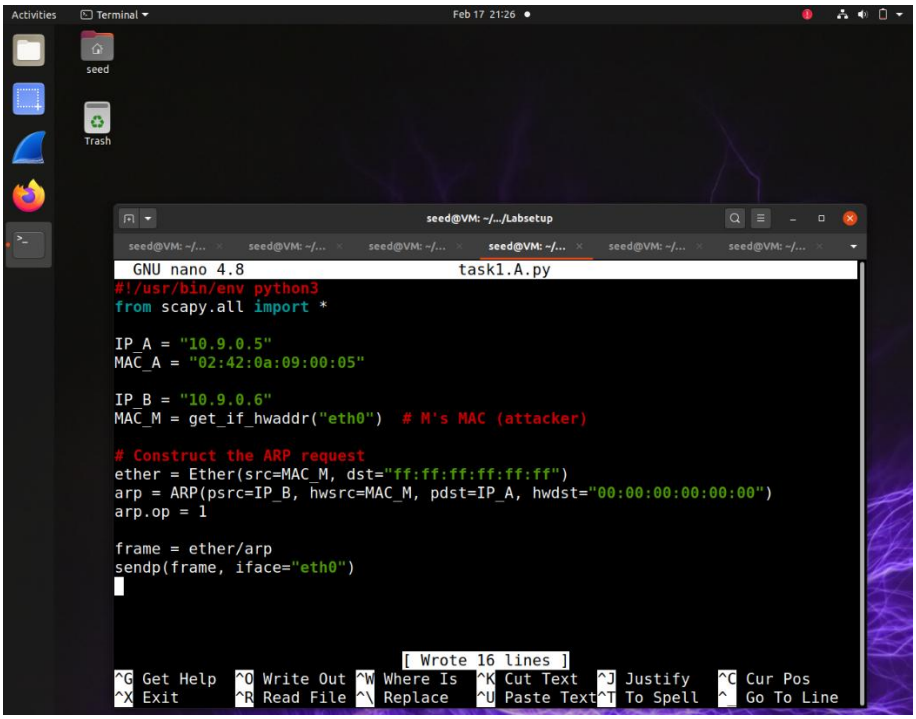
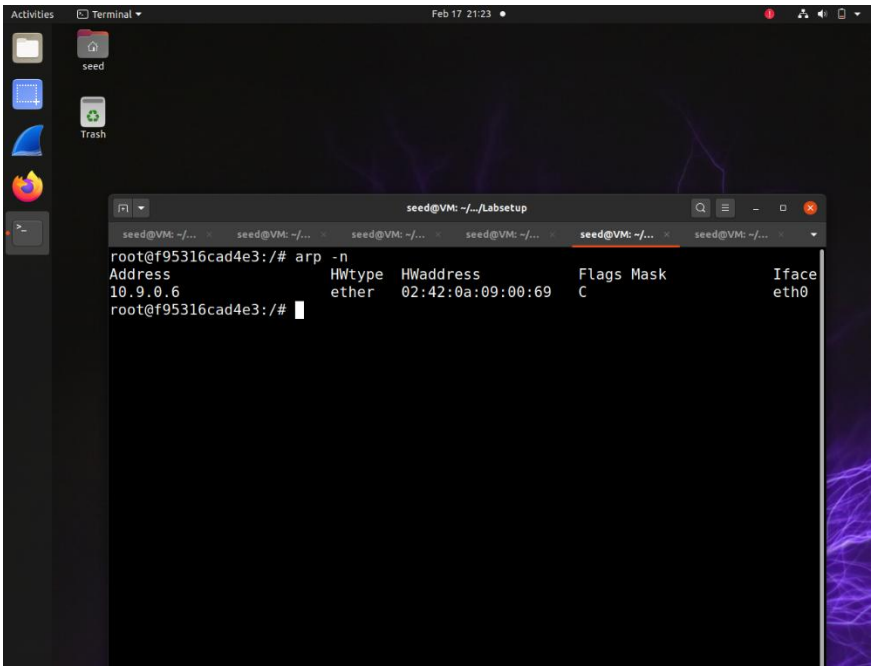
Brandon Sardi

ARP Cache Poisoning Attack

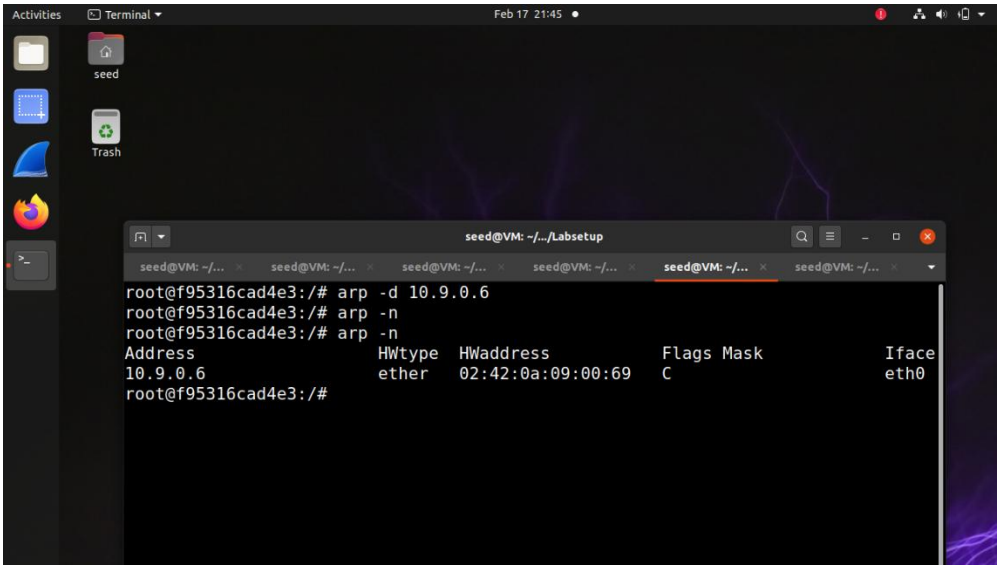
Task1.A



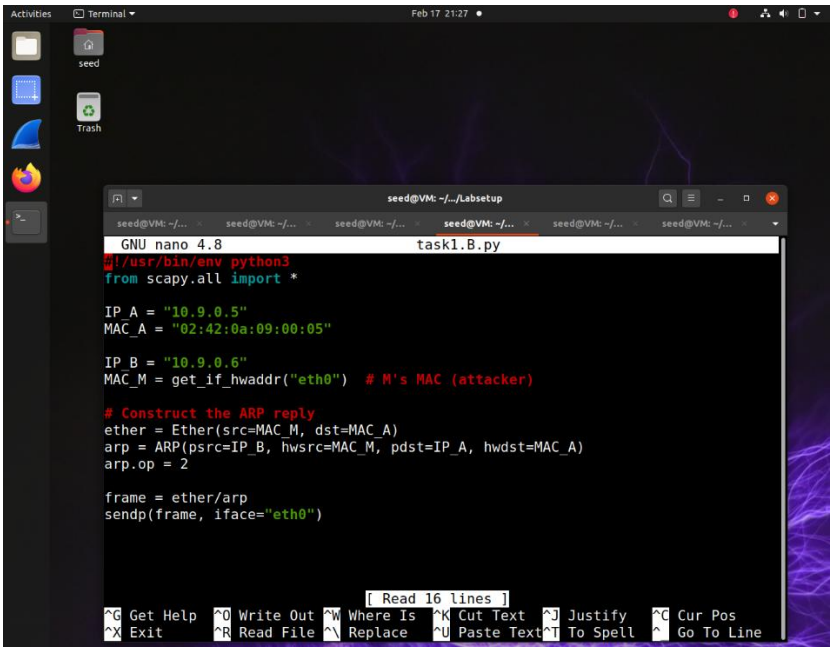
Here is the code for task1.A, I assign the victims IP and MAC address to variables as well as the IP of host B and the attackers MAC as well. I then construct a spoofed ARP request from M to A



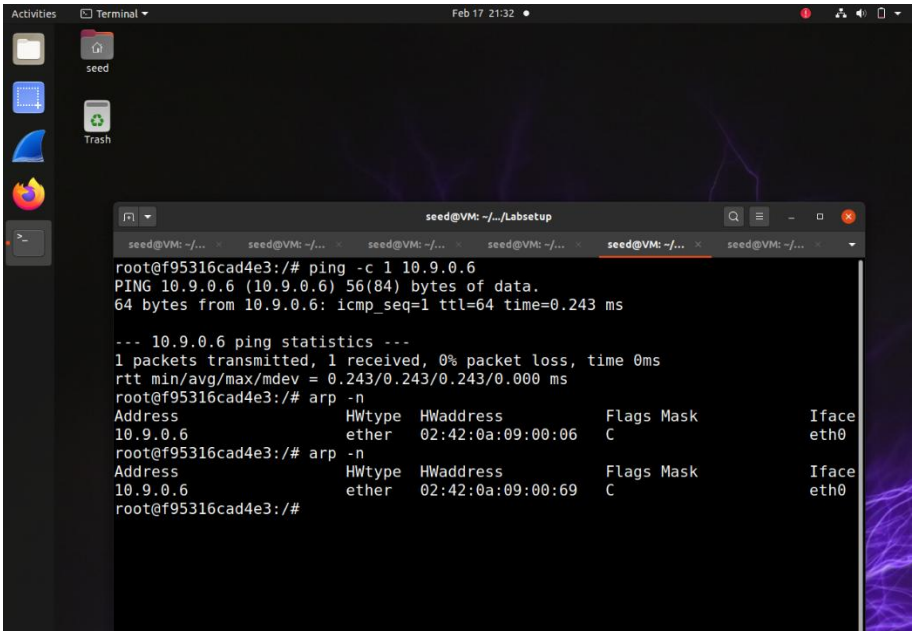
Here is the result of the ARP cache on host A, this ARP cache poisoning attack works in both scenarios, if host B's IP is already in host A's cache and if B's IP is not in A's cache.



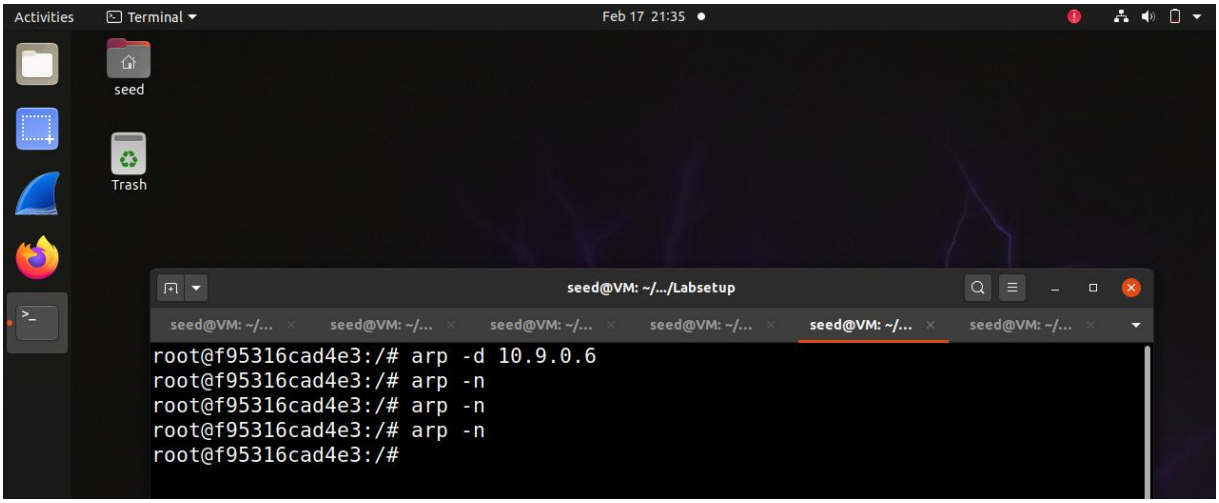
Task1.B



Here the main difference in the code is that we are doing arp.op=2, which is the ARP op code for ARP reply instead of request. So we are sending a spoofed reply from M to A saying 10.9.0.6 is at M's MAC.

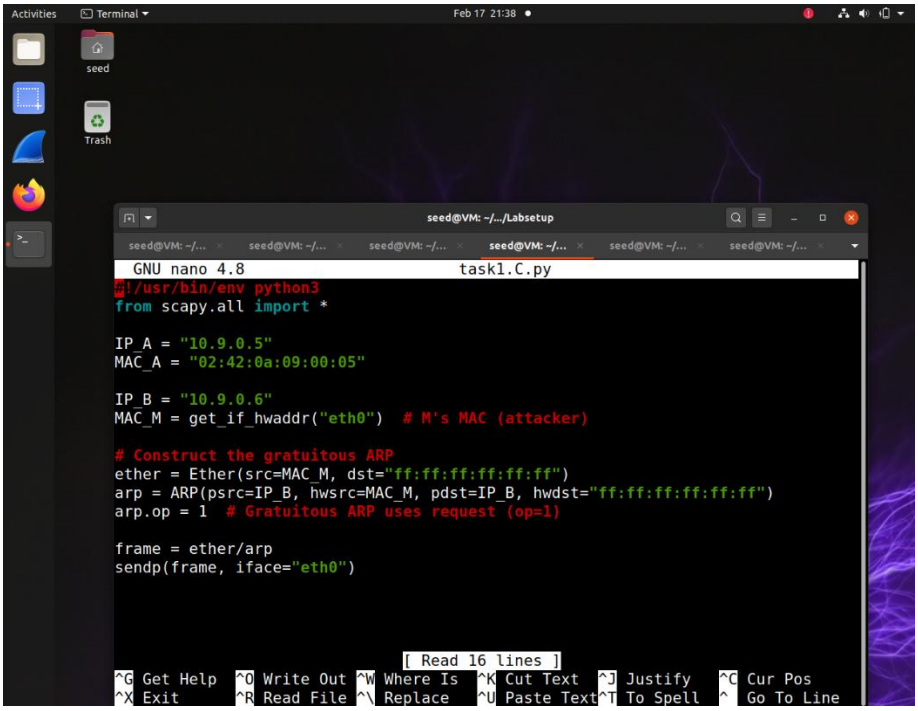


We can see here that it was a success if B's IP was already in A's cache, updating B with M's MAC.

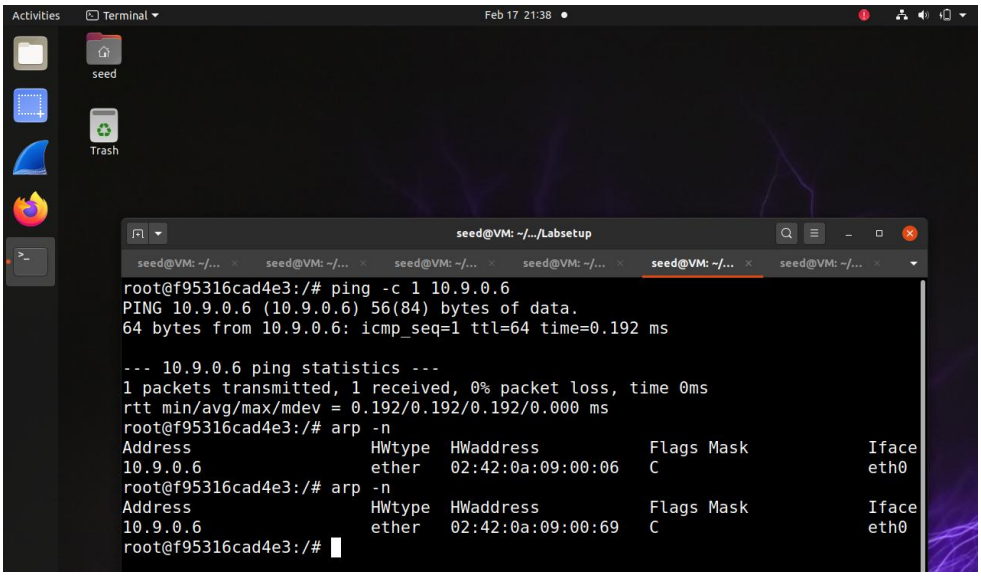


Here we can see that the spoofed reply failed with B's IP not in A's cache so no entry was added. ARP replies only work when updating existing entries.

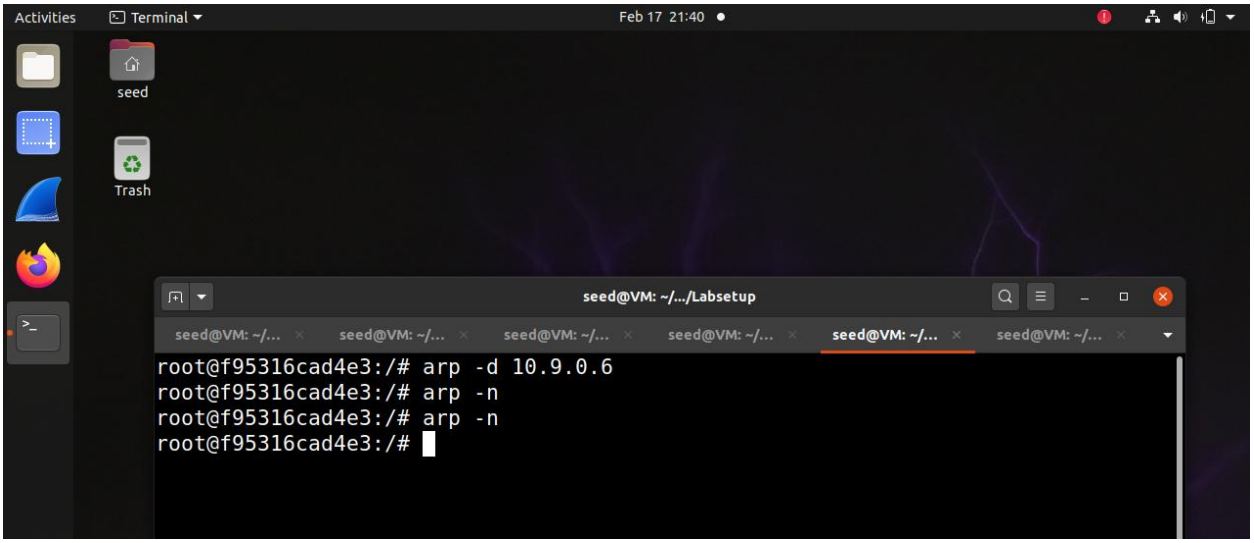
Task1.C



Here we are sending a gratuitous ARP packet where the source and destination IP's are the same.



We can see that it is a success with B's IP already in A's cache.

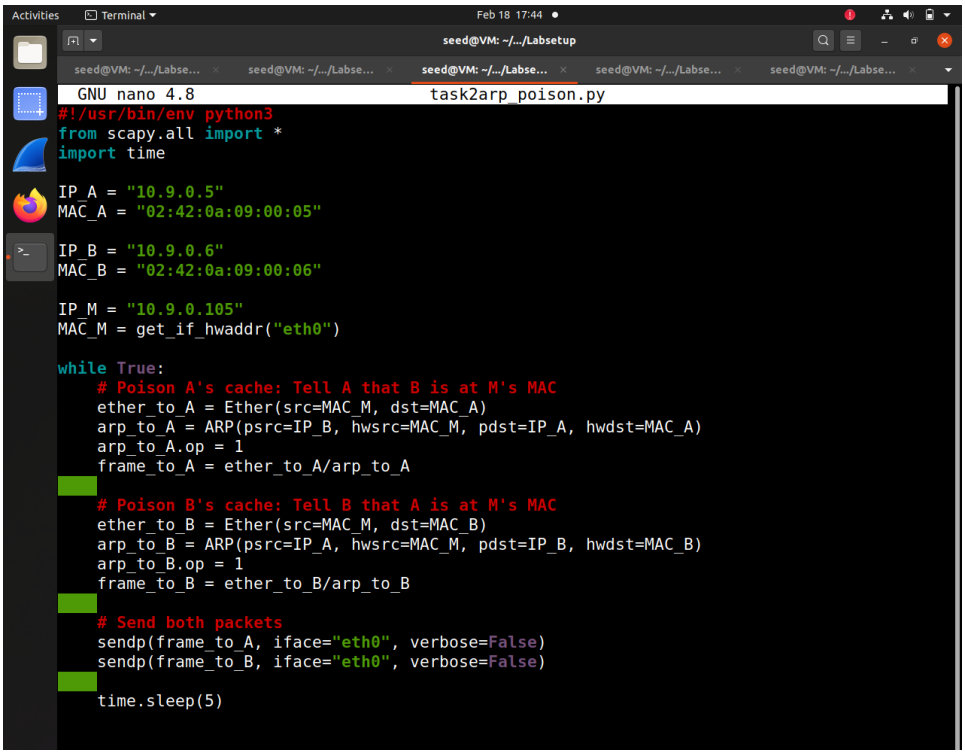


However, it once again fails when B's IP is not in A's cache, no entry was added. After some research I found out that gratuitous ARP is used for hosts to announce or update their IP to MAC mappings, however like unsolicited ARP replies, modern systems only accept gratuitous ARP for updating existing entries.

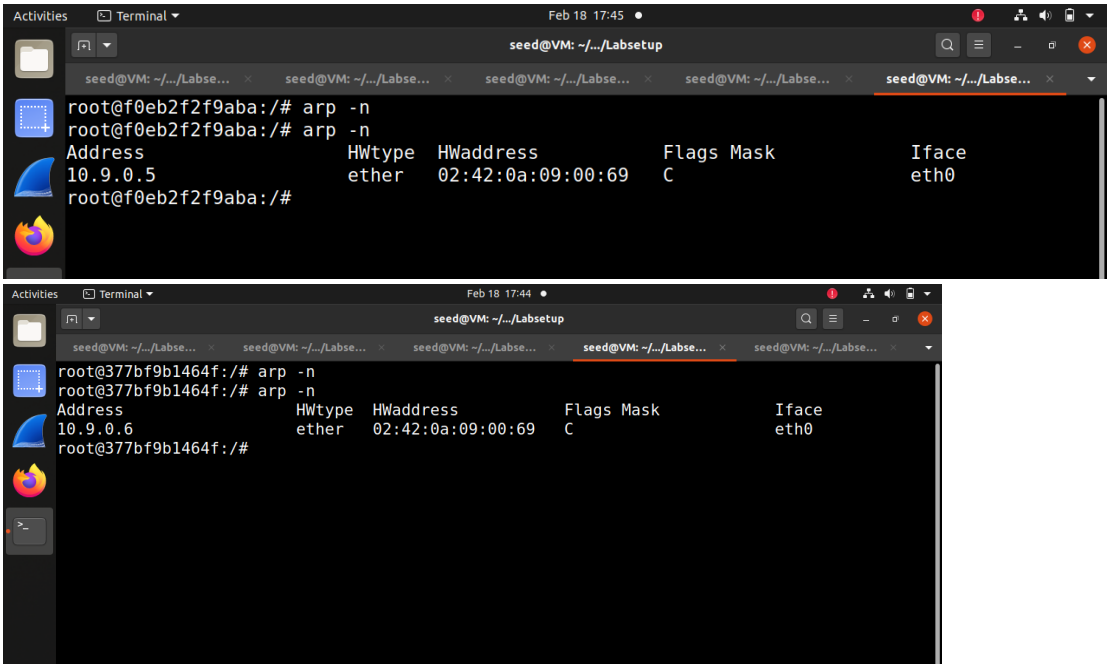
Task 2

The goal of Task 2 was to perform a Man-in-the-Middle (MITM) attack on a Telnet session between Host A (client) and Host B (server) using ARP cache poisoning. By poisoning both A and B's ARP caches, we redirected their traffic through Host M (attacker), allowing us to intercept and modify the Telnet data in transit.

Step 1

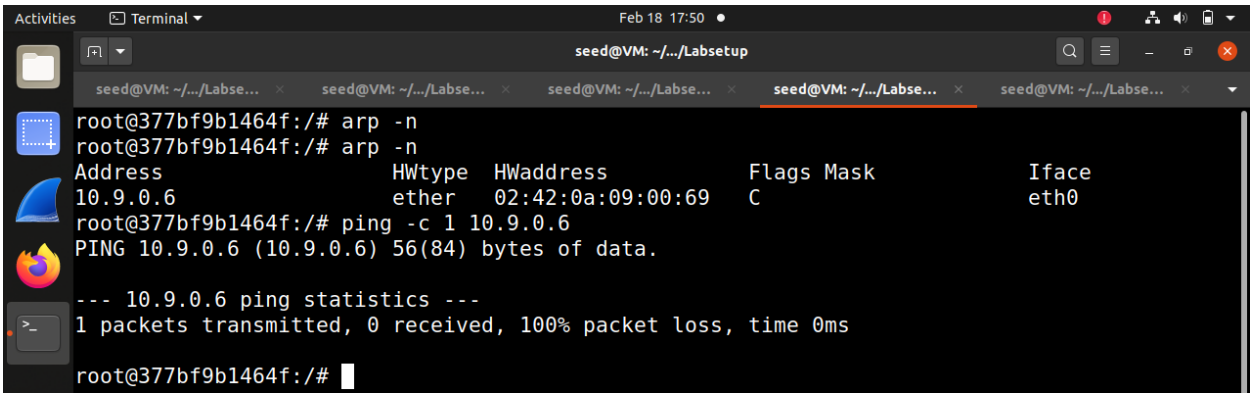


To maintain the MITM position, we needed to continuously poison both A and B's ARP caches every 5 seconds. This prevents the real ARP entries from being restored.

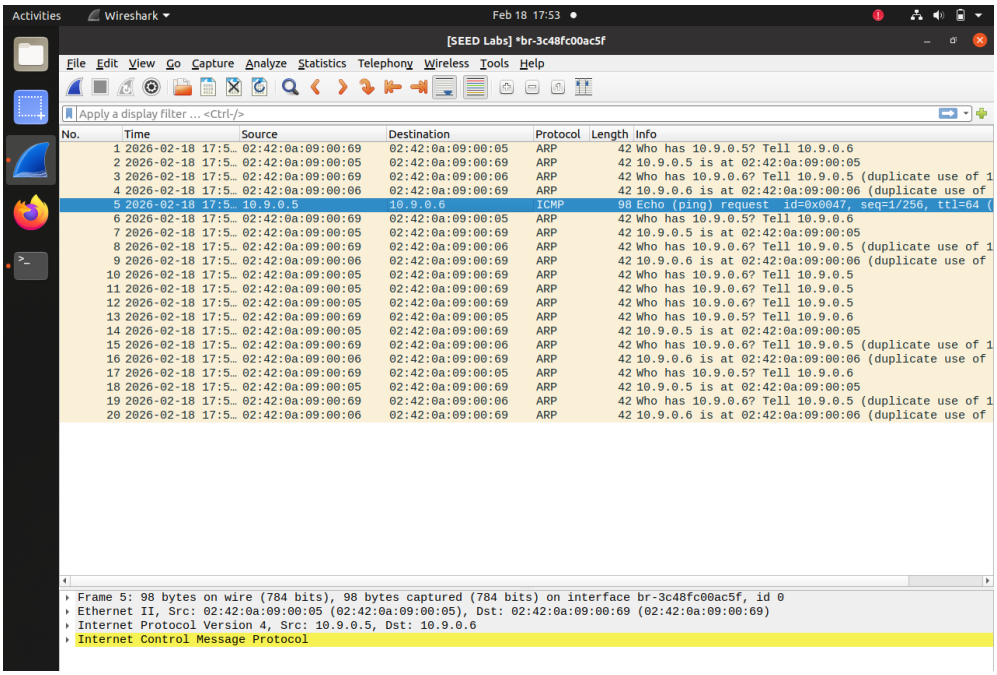


After running the ARP poison attack, we can see it updated the host's ARP cache on both machines.

Step 2

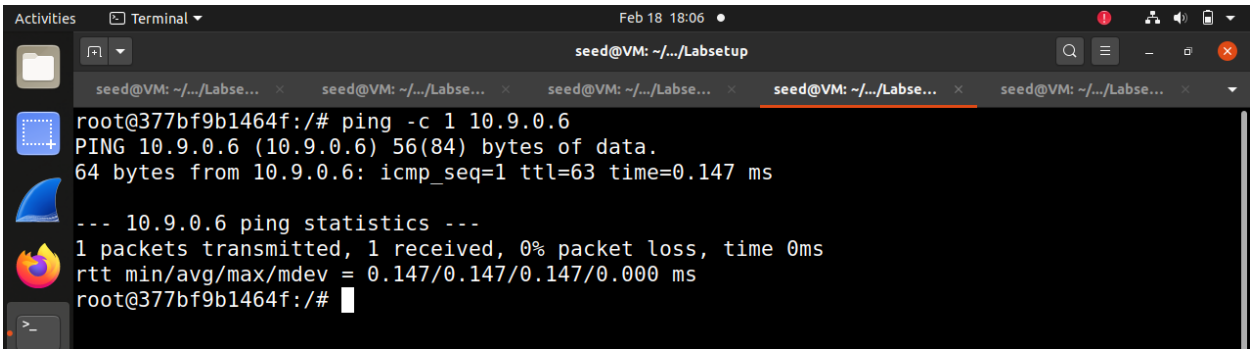


After turning off IP forwarding, we can see here that the ping to host B failed with 100% packet loss, this is because A's poisoned cache made it send packets to M instead of B so B never received the packets.

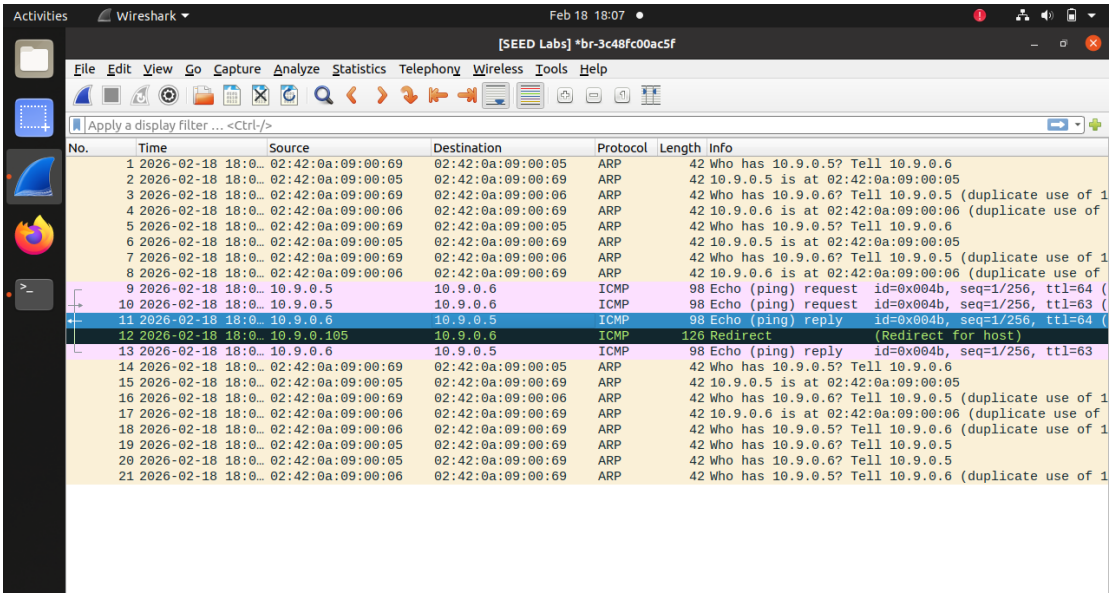


Here is the Wireshark result that shows the continuous ARP poisoning as well as the ICMP echo request from A to B with no ICMP echo reply.

Step 3

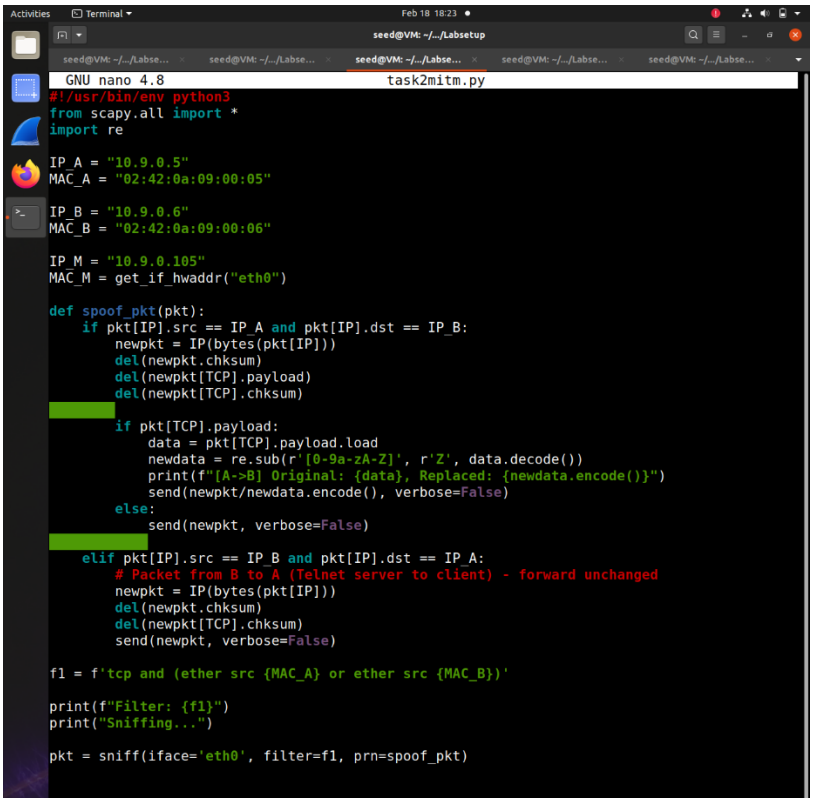


Now, after turning IP forwarding back on we can see that the ping now worked. Host A sends packets to M thinking its B, M receives and forwards them to the real B, B responds to M thinking it's a and then M forwards the response back to A.

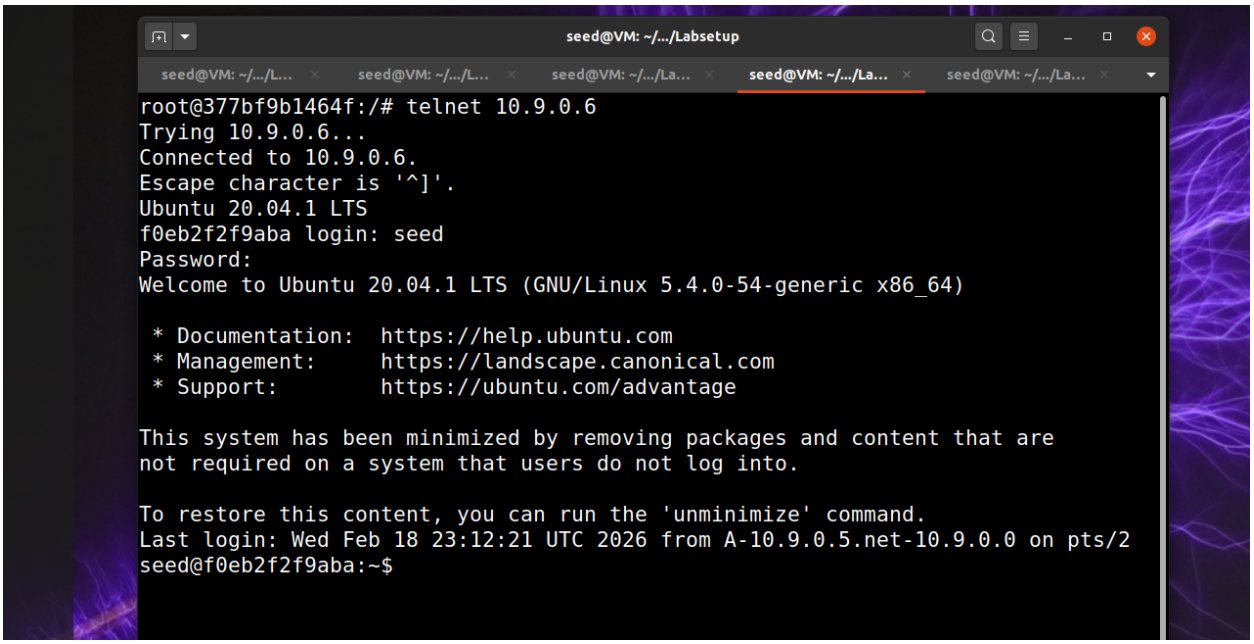


We can see here the ICMP echo request and the ICMP echo reply.

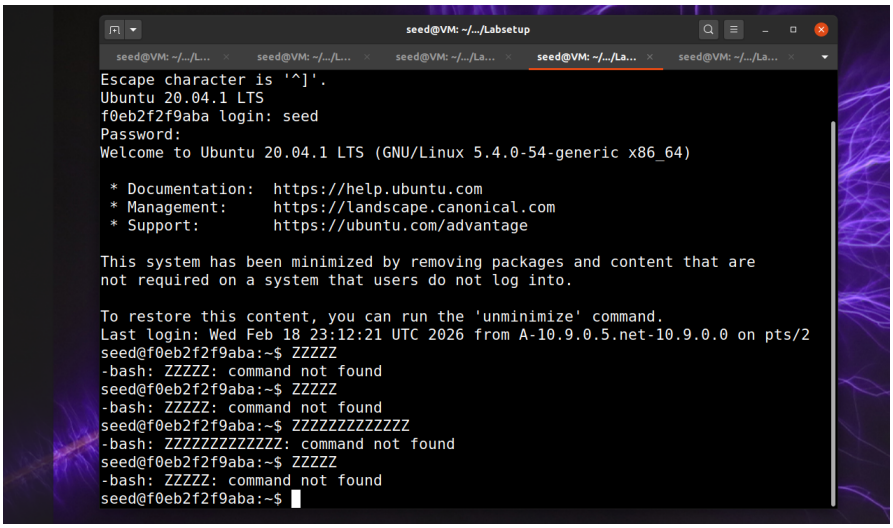
Step 4



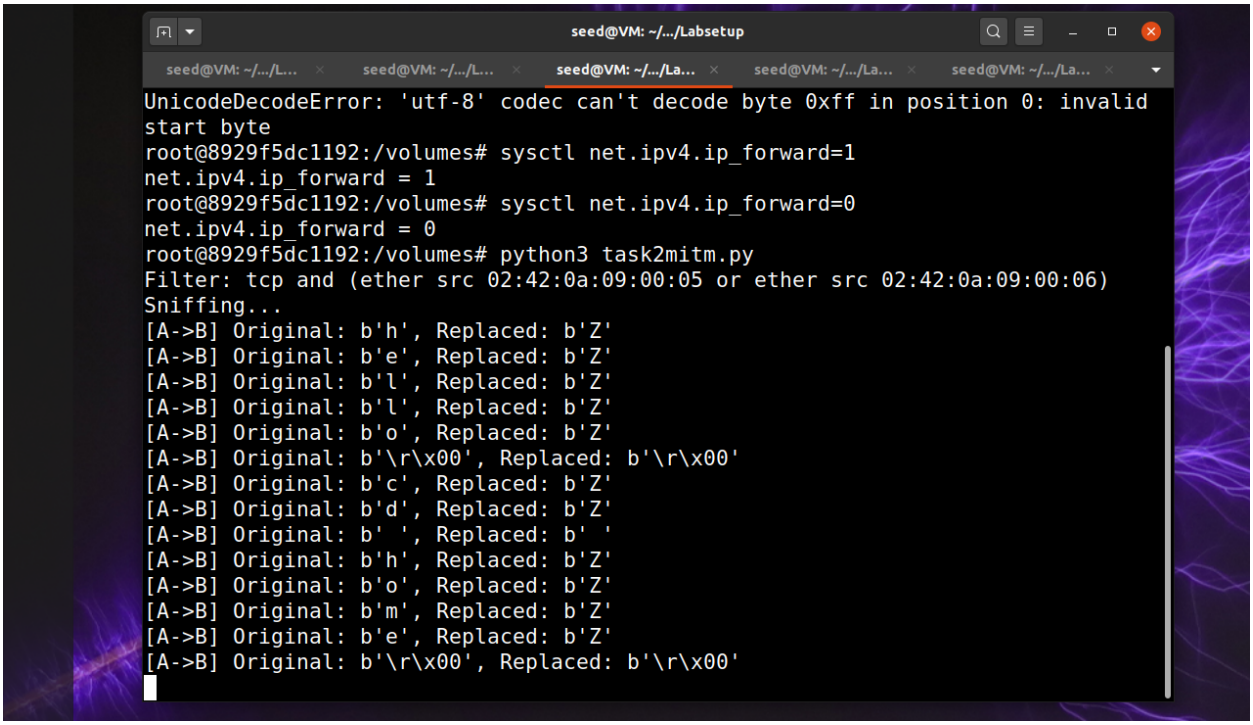
So in this step we have our ARP poisoning python program running in the background with IP forwarding on, then we have host A connect to host B via telnet, after they are connected on the attacker machine, I turn off port forwarding.



When IP forwarding was turned off, I noticed that the telnet window was producing no output, the characters being typed were sent to M but not forwarded to B, the telnet session appeared to be frozen.

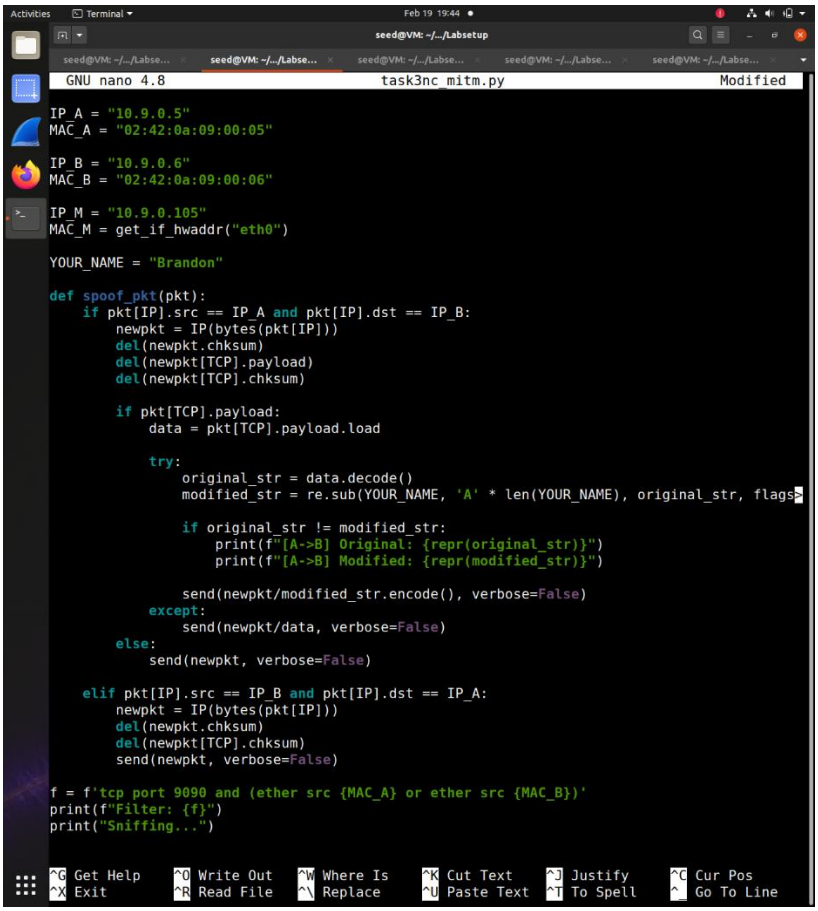


After running the MITM python program on the attacker's machine (M) I see that the output on the telnet screen is always showing Z instead of what I am actually typing. Even when typing exit the command wouldn't work until after I turned back on IP forwarding.

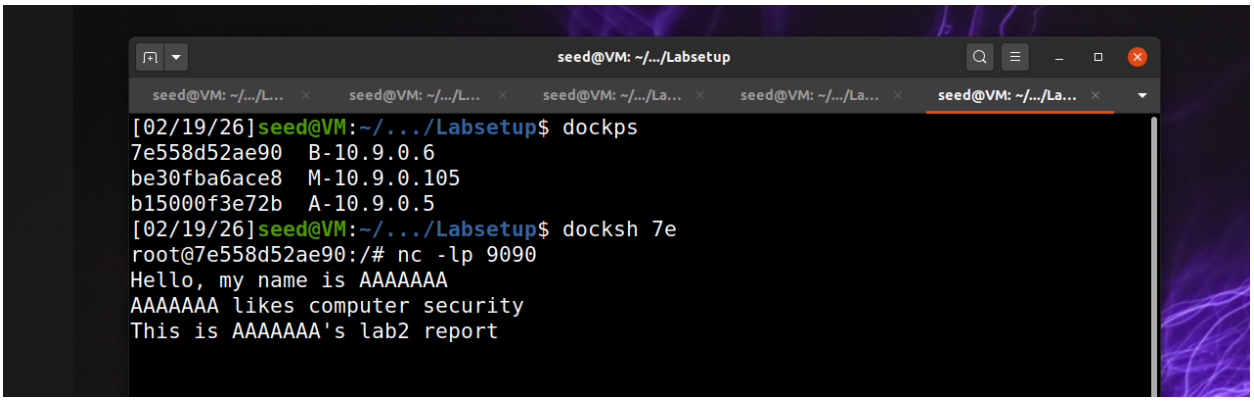


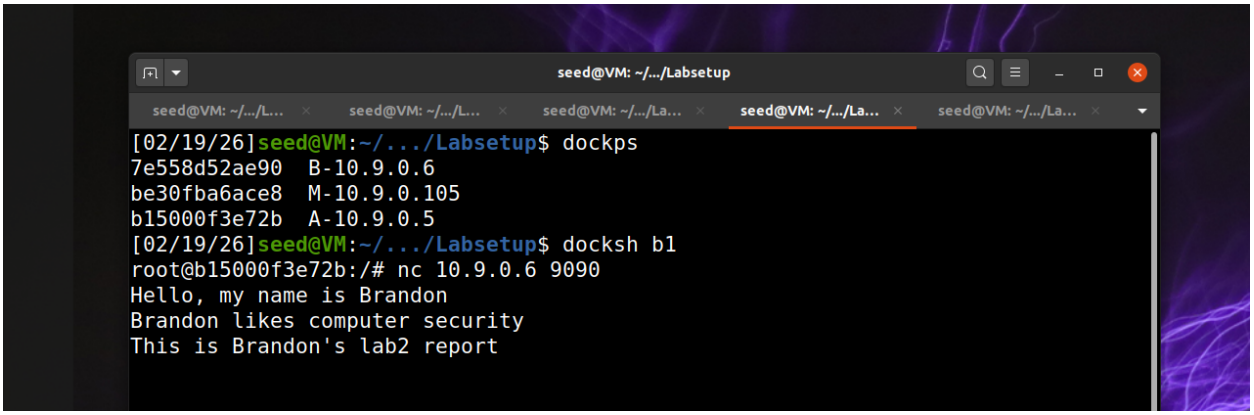
On the attacker's machine (M) we can see the actual keys being pressed by host B.

Task 3



The goal of Task 3 was to perform a Man-in-the-Middle (MITM) attack on a netcat TCP connection between Host A (client) and Host B (server) using ARP cache poisoning. Unlike Task 2 which used Telnet, this task required intercepting and modifying netcat messages to replace occurrences of a specific name (my first name) with an equal number of 'A' characters while preserving TCP sequence numbers.





As you can see in these 2 screenshots the attack was successful, it changed my name Brandon to only display as A's on host A.